

Задание 1. Простая криптографическая библиотека

1. Цель

Создать базовую динамическую библиотеку `libcaesar` для шифрования данных с использованием побайтовой операции XOR. Данная библиотека станет основой для последующих заданий курса и познакомит студентов с принципами создания библиотек в среде Linux, работой с компилятором GCC, использованием Makefile для автоматизации сборки, а также механизмом динамической загрузки библиотек во внешних приложениях.

2. Описание

В рамках данного задания необходимо разработать динамическую библиотеку, реализующую простейший алгоритм шифрования — побайтовое исключаящее ИЛИ (XOR) с ключом. В отличие от классического шифра Цезаря, данный алгоритм обрабатывает данные как поток байт, что позволяет шифровать не только текст, но и любые бинарные файлы.

Программная реализация библиотеки (на языке C или C++) должна включать две обязательные функции с следующими сигнатурами:

1. `void set_key(char key)` — устанавливает ключ шифрования. Ключ должен сохраняться внутри библиотеки и использоваться при последующих вызовах функции шифрования.
2. `void caesar(void* src, void* dst, int len)` — выполняет шифрование/дешифрование. Функция читает `len` байт из буфера `src`, применяет XOR с установленным ключом и записывает результат в буфер `dst`. Буферы `src` и `dst` могут указывать на одну и ту же область памяти.

Поскольку операция XOR симметрична, одна и та же функция `caesar` используется как для шифрования, так и для дешифрования (при условии, что ключ не менялся).

Makefile должен содержать три обязательные цели:

- `all` (или `make`): компиляция библиотеки и создание файла `libcaesar.so`.
- `install`: копирование собранной библиотеки в системную директорию `/usr/local/lib` с обновлением кэша динамических библиотек (`ldconfig`).
- `test`: запуск тестовой консольной программы для демонстрации работы.

Тестовая консольная программа может быть написана на **любом языке программирования** (C, C++, Python, Go и т.д.). Она должна демонстрировать динамическую загрузку библиотеки (не статическую линковку на этапе компиляции). При запуске программа должна принимать 4 аргумента командной строки:

1. Путь к файлу библиотеки (например, `./libcaesar.so`).
2. Ключ шифрования (символ или число).
3. Путь к входному файлу (исходные данные).
4. Путь к выходному файлу (результат).

Программа должна прочитать содержимое входного файла, загрузить библиотеку, установить ключ, вызвать функцию `caesar` и записать результат в выходной файл.

3. Критерии приемки

Студент считается успешно выполнившим задание при соблюдении следующих условий:

Функциональные требования:

- Функция `set_key` корректно сохраняет ключ для последующего использования.
- Функция `caesar` корректно выполняет побайтовый XOR для указанного количества байт.
- Двойное применение функции `caesar` с одним и тем же ключом возвращает исходные данные ($A \text{ XOR } K \text{ XOR } K = A$).
- Библиотека корректно обрабатывает бинарные данные (нулевые байты, непечатные символы).
- Поддерживается случай, когда указатели `src` и `dst` совпадают (шифрование на месте).

Требования к сборке:

- Команда `make` успешно создает файл `libcaesar.so` без ошибок компиляции.
- Команда `make install` копирует библиотеку в `/usr/local/lib/`.
- Команда `make test` выполняется без ошибок: тестовая программа запускается, обрабатывает тестовый файл и создает корректный результат.
- Библиотека является динамической (shared library).

Требования к коду:

- Код библиотеки написан на C или C++.
- Код тестовой программы написан на любом языке, поддерживающем динамическую загрузку.
- Отсутствуют утечки памяти (рекомендуется проверить через `valgrind` для C/C++ реализации).
- Код хорошо структурирован и содержит комментарии.
- Сигнатуры функций в библиотеке точно соответствуют заданию.

4. Обязательные ограничения

Технические ограничения:

- Работа должна выполняться в среде Linux (рекомендуется Ubuntu 20.04 или новее).
- Компиляция библиотеки должна производиться с флагами `-Wall -Wextra -pedantic -fPIC`.
- Библиотека должна использовать соглашение о вызовах C (для C++ файлов обязательно использование `extern "C"`).
- Функции должны быть экспортированы (не быть `static` внутри единицы компиляции).

Ограничения по реализации:

- Запрещается использовать внешние криптографические библиотеки (OpenSSL и т.п.).
- Ключ шифрования должен храниться внутри библиотеки (в статической переменной или аналоге), так как он устанавливается отдельно от процесса шифрования.
- Функция `caesar` не должна выделять память внутри себя (память для `dst` предоставляется вызывающей стороной).
- Тестовая программа должна загружать библиотеку динамически во время выполнения (например, через `dlopen / dlsym` в C или `ctypes / cffi` в Python), используя путь, переданный в аргументах.

Организационные ограничения:

- Все файлы проекта (исходники библиотеки, Makefile, тестовая программа, тестовые файлы) должны находиться в одной директории (или поддиректориях проекта).
- При сдаче необходимо продемонстрировать работоспособность всех трёх целей Makefile.
- Тестовый запуск (`make test`) должен использовать заранее подготовленные тестовые данные (например, текстовый файл `input.txt`), чтобы результат можно было проверить автоматически или визуально.